



Advanced Programming

Input / Output Streams

File I/O

The Context

- How do we **read from a file**, or **write to a file**:
 - *text*: characters or lines of text, ...?
 - *binary*: arrays of bytes ...?
- How do we **send or receive**:
 - objects in a distributed application?
 - data to/from a serial port?
- How do threads communicate asynchronous?
- How do we write very large sets of data into a relational database?
- ...

I/O Streams

- A stream is **a sequence of data**: serial, unidirectional, having a source / destination.
- **Byte Streams (8 bits)** / **Char Streams(16 bits)**
- *Producers (Output Streams)*



- *Consumers (Input Streams)*



Using I/O Streams

```
import java.io.*;

open the stream; //new StreamClass([arguments]);

while (there is more data to read or write) {

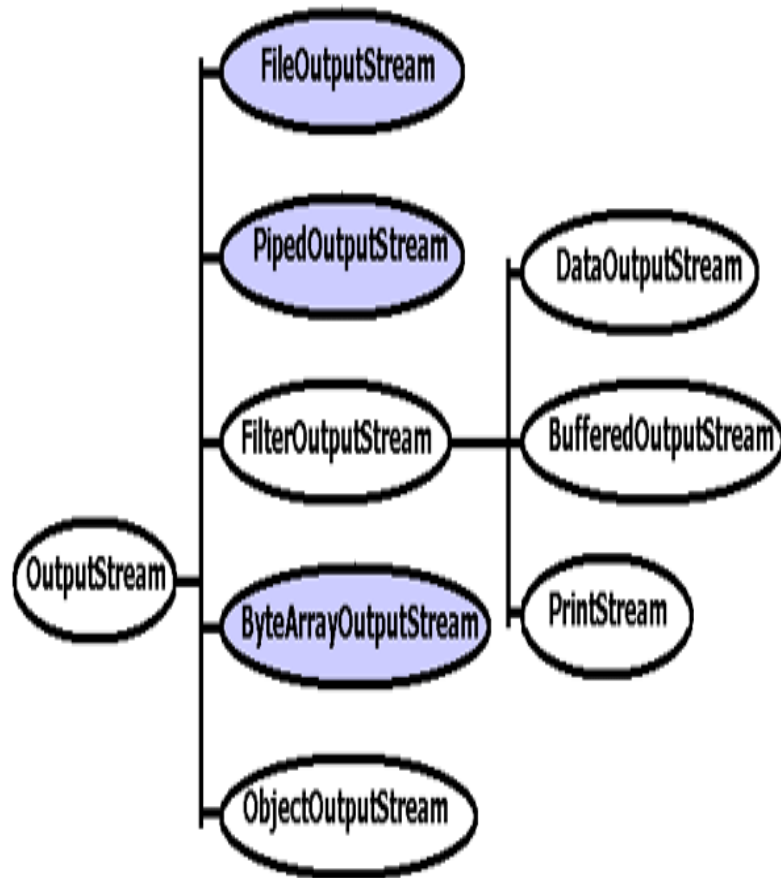
    read/write some data;

    //invoke read(), write() or other specialized methods
}

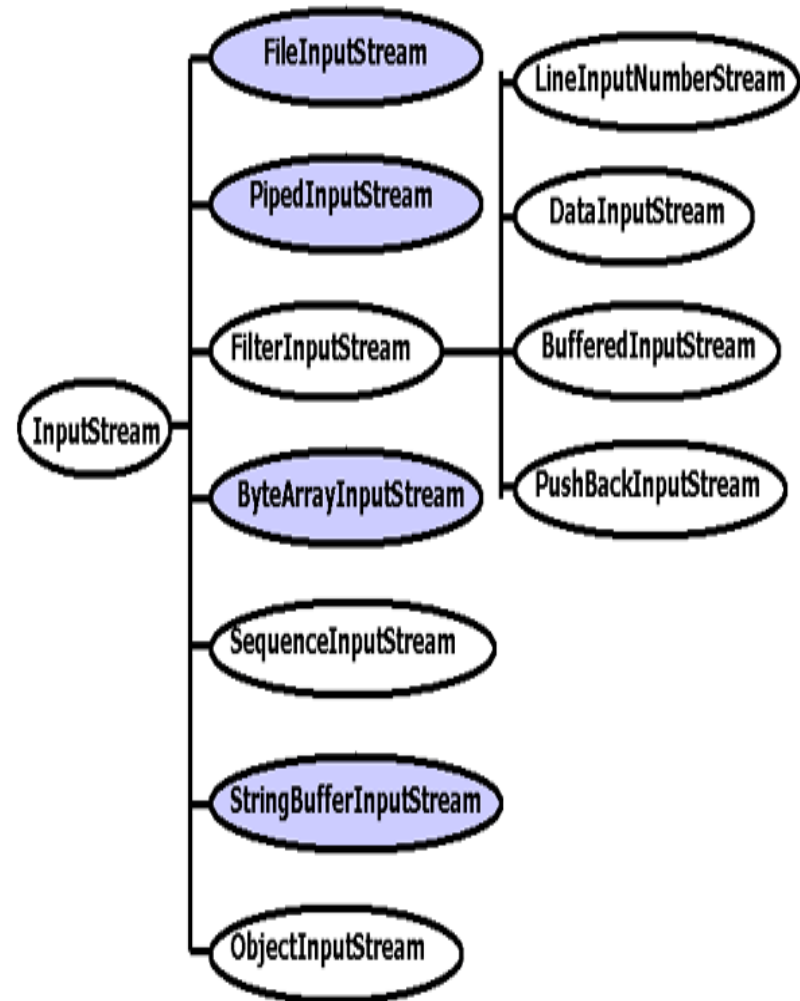
close the stream; //...dont't forget to invoke close()

take care of the possible exceptions: IOException;
```

Byte Streams

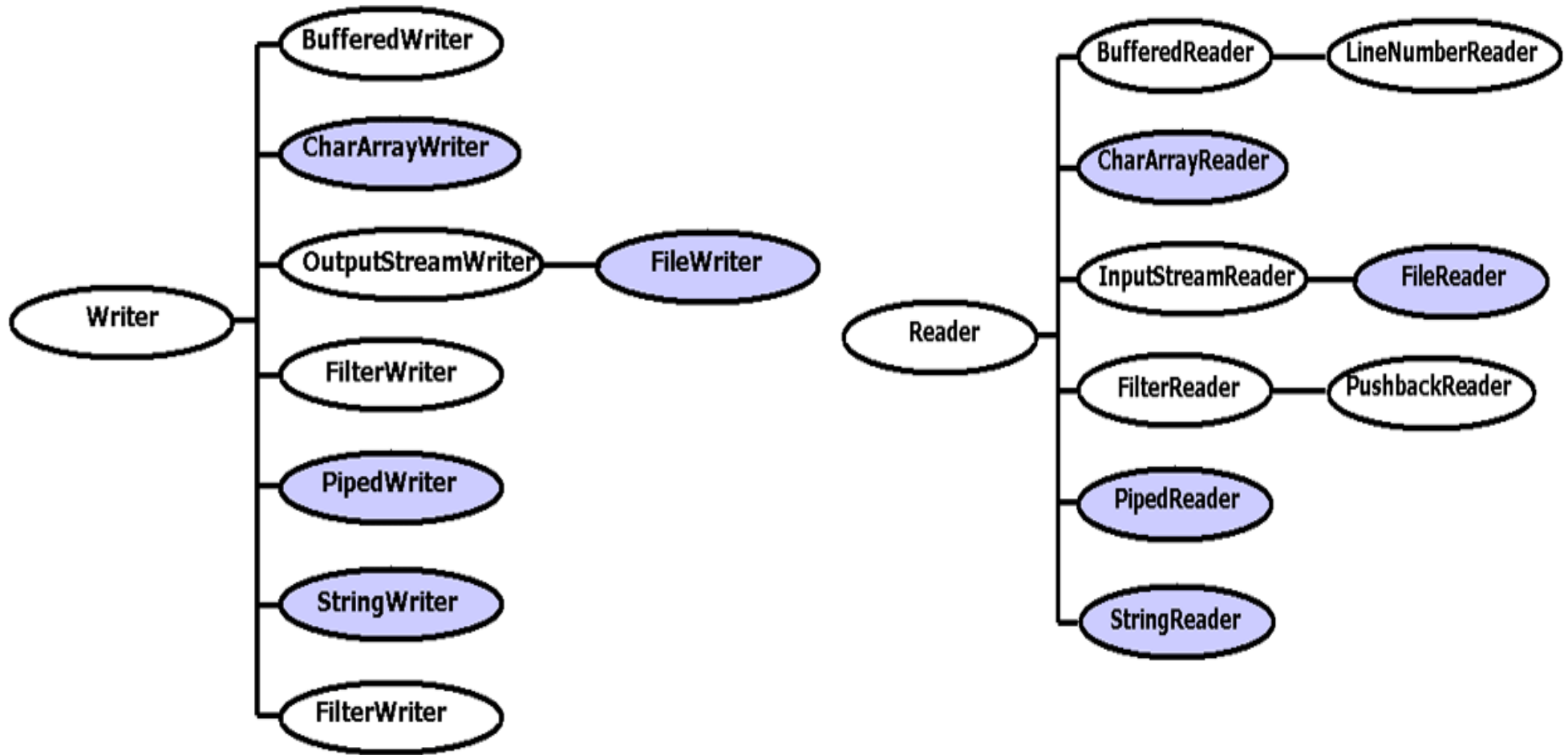


OutputStream = the superclass of all classes representing an **output stream of bytes**. Applications that need to define a subclass of **OutputStream** must always provide at least a method that writes one byte of output



InputStream = the superclass of all classes representing an **input stream of bytes**. Applications that need to define a subclass of **InputStream** must always provide a method that returns the next byte of input

Character Streams



Writer = abstract class for writing to character streams.

Reader = abstract class for reading character streams.

Primitive Streams vs. Decorators

- *Primitive* streams “know” to effectively communicate (read or write) with an external “partner” (file, memory, thread, etc.). Examples:
 - `FileReader, FileWriter`
 - `ByteArrayInputStream, ByteArrayOutputStream`
 - `PipedReader, PipedWriter`
- *Decorator* streams “know” to communicate with another stream (primitive or not) in order to process the raw data and offer specialized methods:
 - `BufferedReader, BufferedWriter, PrintWriter`
 - `DataInputStream, DataOutputStream`
 - `ObjectInputStream, ObjectOutputStream`

Creating Streams

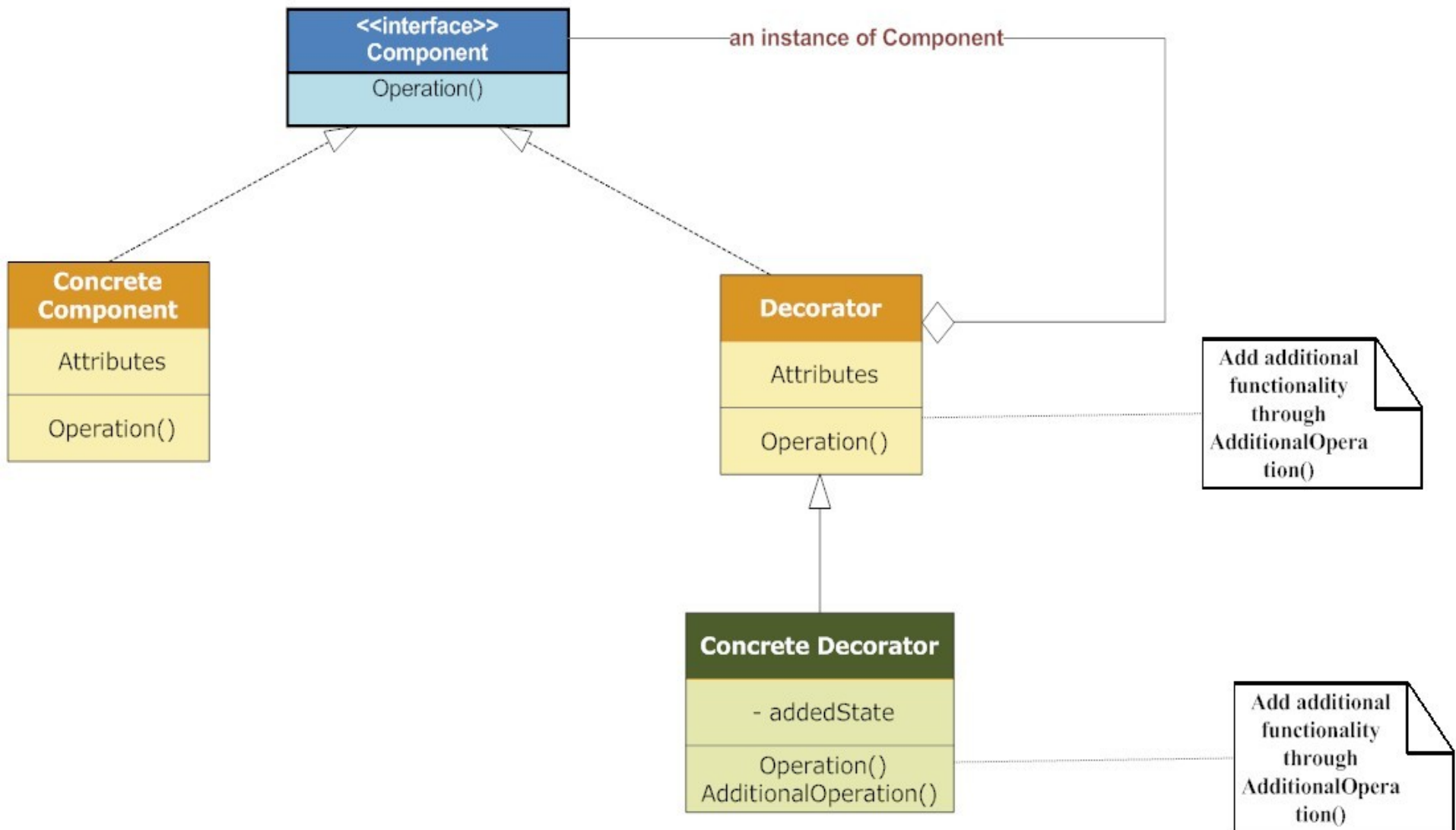
```
PrimitiveStream stream =  
    new PrimitiveStream(externalResource) ;  
DecoratorStream decorator =  
    new DecoratorStream(stream) ;
```

Exemplu:

```
FileReader fileReader = new FileReader("fichier.txt") ;  
BufferedReader bufferedReader =  
    new BufferedReader(fileReader) ;  
String line = bufferedReader.readLine() ;
```


Decorator Design Pattern

Decorator Pattern Structure



Decorator: *Reader*

- Abstract class for reading character streams. The only methods that a subclass must implement are `read(char[], int, int)` and `close()`. Most subclasses, however, will override some of the methods defined here in order to provide higher efficiency, additional functionality, or both.

```
public abstract class Reader implements Readable, Closeable {  
  
    ...  
  
    public abstract int read(char cbuf[], int off, int len)  
        throws IOException;  
  
    public abstract void close() throws IOException;  
  
    ...  
  
}
```

Decorator: *BufferedReader*

- Reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines..

```
public class BufferedReader extends Reader { {  
    private Reader in; //the decorated object  
    public BufferedReader(Reader in) {  
        this.in = in;  
        ...  
    }  
    public String readLine() throws IOException {  
        ...  
        in.read( ... );  
        ...  
    }  
}
```

Buffered Streams

`BufferedReader, BufferedWriter, BufferedInputStream, BufferedOutputStream`

Read /Write data from a stream, buffering elements so as to provide for the **efficient reading of arrays, and lines.**

```
BufferedOutputStream out = new BufferedOutputStream(  
    new FileOutputStream("out.dat"), 1024)  
    //1024 is the size of the buffer  
  
for(int i=0; i<100; i++) {  
    out.write(i);  
    //the buffer is not full yet, the file contains nothing  
}  
  
out.flush();  
//the buffer is flushed, data is written to the file
```

- Greatly reduces the access to the external resource
- Increases the execution speed

Testing the *BufferedWriter*

```
public class TestBufferedWriter {

    private void test(Writer writer) throws IOException {
        try (writer) {
            long t0 = System.currentTimeMillis();
            int n = 20_000_000;
            String test = "Lorem ipsum dolor sit amet";
            for (int i = 0; i < n; i++) {
                writer.write(test);
            }
            System.out.println((System.currentTimeMillis() - t0) + " ms");
        }

    }

    public static void main(String args[]) throws IOException {
        var app = new TestBufferedWriter();
        app.test(new FileWriter("d:/test.txt")); //14s
        app.test(new BufferedWriter(new FileWriter("d:/test.txt"))); //5s
    }
}
```

Serializing Primitive Data

`DataInputStream, DataOutputStream`

- ❖ Writing/Reading primitive data in binary format, “*in a machine-independent way*”

//Writing

```
FileOutputStream fos = new FileOutputStream("test.dat");  
DataOutputStream out = new DataOutputStream(fos);  
out.writeInt(12345);  
out.writeDouble(12.345);  
out.writeBoolean(true);  
out.writeUTF("Sir de caractere");  
out.flush();  
fos.close();
```

...

//Reading

```
FileInputStream fis = new FileInputStream("test.dat");  
DataInputStream in = new DataInputStream(fis);  
int i = in.readInt();  
double d = in.readDouble();  
boolean b = in.readBoolean();  
String s = in.readUTF();  
fis.close();
```

Serializing Objects

`ObjectInputStream, ObjectOutputStream`

- ❖ Writing/Reading objects in a binary format, *“in a machine-independent way”*

//Writing

```
FileOutputStream fos = new FileOutputStream("test.ser");  
ObjectOutputStream out = new ObjectOutputStream(fos);  
out.writeObject(new Date());  
out.writeObject("Hello World");  
out.writeInt(12345);  
out.flush();  
fos.close();
```

//Reading

```
FileInputStream fis = new FileInputStream("test.ser");  
ObjectInputStream in = new ObjectInputStream(fis);  
Date date = (Date)in.readObject();  
String message = (String)in.readObject();  
int i = in.readInt();  
fis.close();
```

Standard I/O Streams

- `System.in` - `InputStream`
- `System.out` - `PrintStream`
- `System.err` - `PrintStream`

Redirecting the standard streams:

`setIn(InputStream)` - redirecting the input
`setOut(PrintStream)` - redirecting the output
`setErr(PrintStream)` - redirecting the error stream

Example:

```
PrintStream fis = new PrintStream(new FileOutputStream("results.txt"));  
System.setOut(fis);  
PrintStream fis = new PrintStream(new FileOutputStream("errors.txt"));  
System.setErr(fis);
```


Scanning and Formatting

```
java.util.Scanner,  
java.util.Formatter, java.text.Format
```

```
Scanner s = Scanner.create(System.in);
```

```
String name = s.next();
```

```
int age = s.nextInt();
```

```
double salary = s.nextDouble();
```

```
s.close();
```

```
System.out.printf("%s %8.2f %n",name, salary, age);
```

```
SimpleDateFormat dateFormat =
```

```
    new SimpleDateFormat("dd-MM-yyyy");
```

```
String date = dateFormat.format(today);
```

```
System.out.println("Today in dd-MM-yyyy format : " + date);
```

java.io.File

- An abstract representation of file and directory pathnames.
- User interfaces and operating systems use *system-dependent* pathname strings to name files and directories.
 - File.separator: \ or /
 - File.pathSeparator ; or :
- Example:

```
File file = new File("file.txt"); //immutable
file.createNewFile(); //→ IOException
file.delete();
```

```
File dir = new File("d:/books");
File[] files = dir.listFiles();
```

“Useful” I/O Classes

- *java.io.File*

An abstract representation of file and directory pathnames.

- *java.io.RandomAccessFile*

Supports both reading and writing to a file, using a file pointer.

- *java.io.StreamTokenizer*

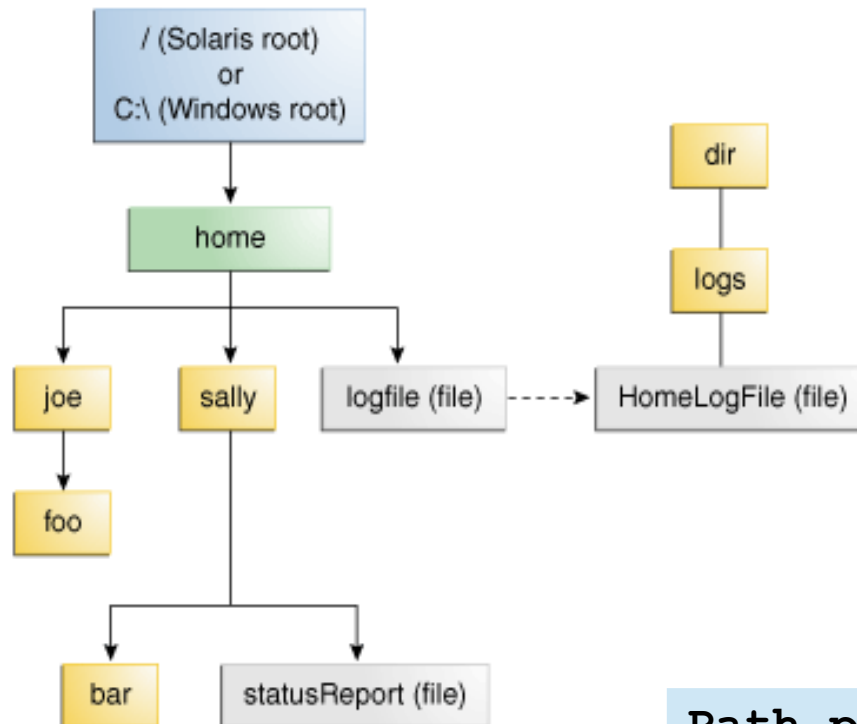
Takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time.

(see also *StringTokenizer*, *String.split*)

- ... here comes the New Java I/O → *java.nio*

File I/O (Featuring NIO)

- *java.io.File* has some “issues” (some methods didn't throw exceptions when they failed, no support for symbolic links, no consistency across platforms, not scalable, no support for recursively walk a file tree, etc.)
- *java.nio.file.Path* is the modern replacement of *File*.



A *Path* object is a programmatic representation of **a path in the file system**. May be:

- relative
- absolute
- symbolic (soft) link
- file
- directory (folder)
- might not exist

```
Path p1 = Paths.get("/home/joe/foo");
```

//may throw InvalidPathException

File Operations

- *java.nio.file.Files* - Static methods that operate on files, directories, or other types of files.
- Examples:

```
boolean readable = Files.isReadable(file);
Files.copy(source, target, REPLACE_EXISTING);
Files.move(source, target, ATOMIC_MOVE);
Files.delete(path);
Path newFile = Files.createFile(path);
Path newDir = Files.createDirectory(path);

BasicFileAttributes attr =
    Files.readAttributes(file, BasicFileAttributes.class);
System.out.println("lastAccessTime: " + attr.lastAccessTime());
...
```

Traversing a Directory Tree

```
Files.walkFileTree(path, new FileVisitor<Path>() { ... });
```

```
Files.walkFileTree(path, new FileVisitor<Path>() {  
    public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes  
attrs) throws IOException {  
        System.out.println("pre visit dir:" + dir);  
        return FileVisitResult.CONTINUE;  
        // or TERMINATE or SKIP_SIBLINGS or SKIP_SUBTREE  
    }  
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)  
throws IOException {  
        System.out.println("visit file: " + file);  
        return FileVisitResult.CONTINUE;  
    }  
    public FileVisitResult visitFileFailed(Path file, IOException exc)  
throws IOException {  
        System.out.println("visit file failed: " + file);  
        return FileVisitResult.CONTINUE;  
    }  
    public FileVisitResult postVisitDirectory(Path dir, IOException exc)  
throws IOException {  
        System.out.println("post visit directory: " + dir);  
        return FileVisitResult.CONTINUE;  
    }  
});
```

Reading and Writing

- Commonly Used Methods for Small Files

```
byte[] fileArray = Files.readAllBytes(path);  
List<String> fileLines = Files.readAllLines(path);  
String fileContent = new String(Files.readAllBytes(path));  
Files.write(path, buffer); // to write bytes, or lines
```

- Buffered I/O Methods for Text Files

```
BufferedReader reader = Files.newBufferedReader(path);  
BufferedWriter writer = Files.newBufferedWriter(path);
```

- Unbuffered Streams

```
InputStream in = Files.newInputStream(path);  
OutputStream out = Files.newOutputStream(path);
```

- Methods for *Channels* and *ByteBuffers*

While stream I/O reads a character at a time, channel I/O reads a buffer at a time

Using Streams and Files

- Iterating over the lines of a text file

```
Path path = Paths.get("c:\\data\\SomeFile.txt");  
try (Stream<String> lines = Files.lines(path)) {  
    lines.forEachOrdered(line->System.out.println(line));  
} catch (IOException e) {  
    System.err.println(e);  
}
```

- Iterating over the files in a directory

```
Path path = Paths.get("c:\\data");  
Files.list(dir)  
    .filter((Path file) ->  
        file.getFileName().toString().endsWith(".pdf"))  
    .forEach(System.out::println);
```

Lazy

NIO Channels

- A *channel* represents an **open connection to an entity such as a hardware device, a file or a network socket**.
- You can **both read and write** to a Channels. Streams are typically one-way (read or write).
- Channels can be read and written asynchronously.
- Channels always read to, or write from, a **Buffer**.
- Most important *Channel* implementations:
 - **FileChannel**
 - DatagramChannel
 - SocketChannel
 - ServerSocketChannel

Basic FileChannel Example

```
RandomAccessFile file =  
    new RandomAccessFile("my_file.txt", "rw");  
  
FileChannel channel = file.getChannel();  
ByteBuffer buf = ByteBuffer.allocate(2048);  
//A buffer is essentially a block of memory into which you can  
//write data, which you can then later read again  
  
int bytesRead = channel.read(buf);  
while (bytesRead != -1) {  
    System.out.println("Read " + bytesRead);  
    buf.flip();  
    //A buffer has capacity, position, limit  
    //The flip() method switches a Buffer from writing mode to reading  
    //mode. Calling flip() sets the position back to 0, and sets the  
    //limit to where position just was.  
    while(buf.hasRemaining()){  
        System.out.print((char) buf.get());  
    }  
    buf.clear();  
    bytesRead = channel.read(buf);  
}  
file.close();
```

Channel Features

- **Scattering/ Gathering**
 - single channels to multiple buffers
 - multiple buffers to single channel
- Channel to Channel **transfers** (Bulk transfers)
 - copying large amounts of data using direct, efficient OS-level operations, rather than looping byte-by-byte.
- **Memory mapped** files
 - Interacting with a file like it's an array in memory
- **Selectors**
 - determine which channels are ready for reading or writing
 - using a single thread to handle multiple channels
- **Non-blocking** IO Pipelines and Servers

Watching a Directory for Changes

- **Watch Service API**
- **Create and Register**

```
WatchService watcher = FileSystems.getDefault().newWatchService();  
Path dir = Paths.get("Path/To/Watched/Directory");  
dir.register(watcher, ENTRY_CREATE, ENTRY_DELETE, ENTRY_MODIFY);
```

- **Watch...**

```
while (true) {  
    WatchKey key = watcher.take(); // wait for a key to be available  
    for (WatchEvent<?> event : key.pollEvents()) {  
        WatchEvent.Kind<?> kind = event.kind();  
        WatchEvent<Path> ev = (WatchEvent<Path>) event;  
        Path fileName = ev.context();  
  
        System.out.println(kind.name() + ": " + fileName);  
        key.reset();  
    }  
}
```